PROJECT ARCHITECTURE

A. Technical Architecture

The IMovies platform is designed as a scalable, modern Web application built using a layered Model-View-Controller (MVC) architecture. The system decouples the presentation layer, application business logic, and persistent storage layer to maximize maintainability, testability, and security.

a. High-Level Architecture Overview

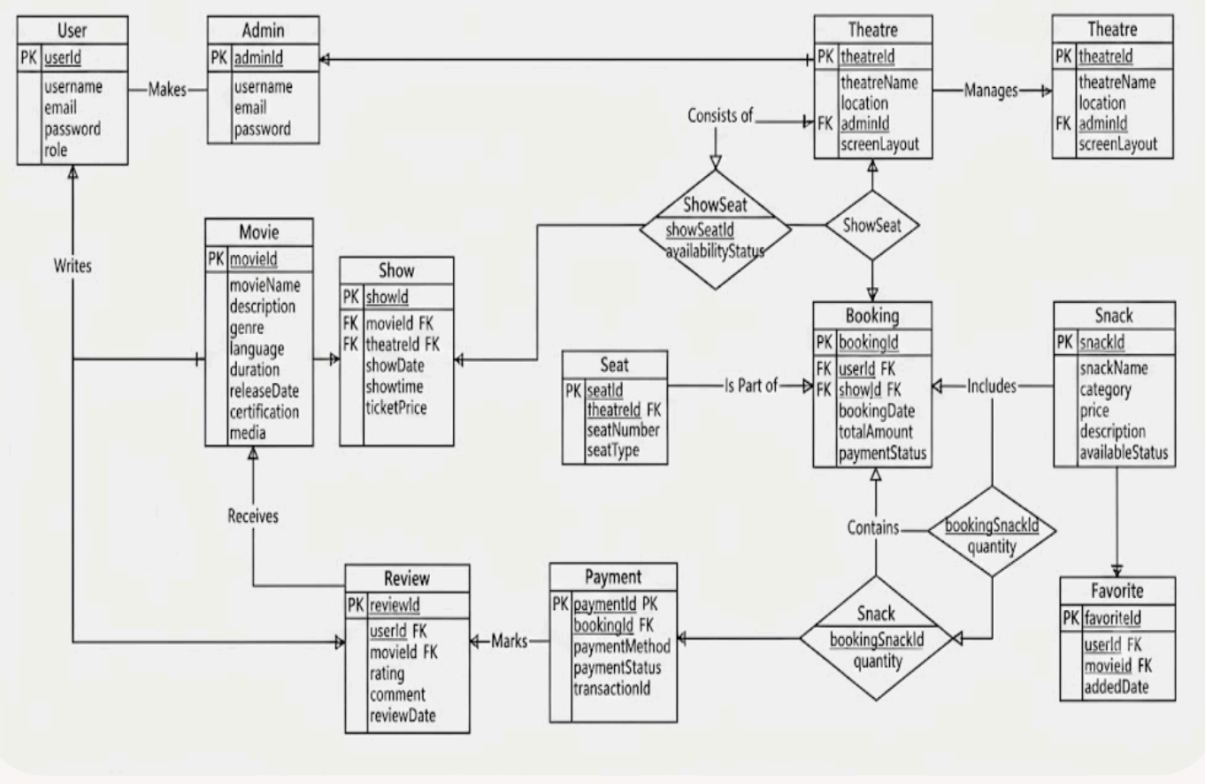
- **Client / Presentation Layer:** Web browser handling render functions via HTML5, CSS3, and JavaScript/Client Frameworks. Communicates via HTTP/HTTPS REST APIs or Server-Rendered Views.
- **Application / Controller Layer:** Core system processing handles request routing, authorization middleware, input validation, and business workflow control.
- **Domain / Business Logic Layer:** Encapsulates service objects, core domain rules, transaction handlers, and third-party integrations (e.g., payment gateways, media transcoding, external movie database APIs).
- **Data Layer:** Object-Relational Mapping (ORM) or Data Access Objects (DAO) interfacing with a relational relational database management system (RDBMS) alongside an in-memory caching layer for performance optimization.

b.Technology Stack Specifications

Layer	Component / Technology	Function
Frontend	React.js / HTML5 / CSS3	Client-side UI rendering, responsive interface, interactive elements
Backend Application	Node.js (Express) / Java (Spring Boot)	Application server, controller logic, RESTful API endpoints
Database	PostgreSQL / MySQL	Relational data persistence, integrity constraints, ACID compliance
Caching	Redis	Session state management, rate limiting, catalog caching

Layer	Component / Technology	Function
File Storage	AWS S3 / Cloud Storage	Storage for movie posters, banners, and media assets
Authentication	OAuth 2.0 / JWT	Secure user identity verification and access token generation

B. ER-Diagram & Explanation



User & Admin:

- **User:** Represents regular customers on the platform (userId as Primary Key). They write reviews (Writes), make bookings, and maintain favorite movie lists.
- **Admin:** Stores administrative credentials and profile details (adminId as Primary Key) to oversee platform operations.

Movie & Show:

- **Movie:** Stores comprehensive movie information including movieId, movie name, description, genre, duration, release date, certification, and media. Users submit reviews for movies (Receives).
- **Show:** Represents individual screening schedules, mapping which movie plays at which theatre on a specific date and time (showId, movieId, theatreId, showDate, showtime).

Theatre & Seat:

- **Theatre:** Stores cinema location details and configurations (theatreId, theatreName, location). Managed by administrators (Manages).
- **Seat:** Tracks specific seating arrangements inside a theatre (seatId, seatNumber, seatType) and is structurally linked to a theatre (Is Part of).

Booking & Snack (Custom Feature):

- **Booking:** Generated when a user reserves tickets, storing bookingId, userId, showId, total amount, and payment status.
- **Snack:** Represents your custom pre-ordering feature for popcorn, beverages, and food items (snackId, snackName, price, availableStatus). The Booking entity connects to snacks (Includes / Contains) so users can pre-order food alongside their ticket reservation.

Payment, Review & Favorite:

- **Payment:** Tracks transaction details (paymentId, paymentMethod, transactionId) associated with each booking.
- **Review:** Stores numerical ratings and written text feedback submitted by users for movies.
- **Favorite:** Maintains users' saved or bookmarked movies for quick reference.

C. Features

Authentication & Authorization

- User Registration and Identity Verification (Email confirmation).
- Secure Role-Based Access Control (RBAC) via JSON Web Tokens (JWT).
- OAuth Integration (Google, Apple Single Sign-On).

Movie Catalog & Management

- Multi-criteria Search & Filtering (by Genre, Release Year, Average Rating, Cast).
- Dynamic Movie Discovery (Trending, Top Rated, New Releases).
- Administrative CRUD (Create, Read, Update, Delete) operations for movie assets.

User Interactivity & Engagement

- User Star-Rating and Text-based Review Submission system.
- Personalized Watchlist curation (Add/Remove/Order titles).
- Social sharing options and recommendation algorithms based on view history.

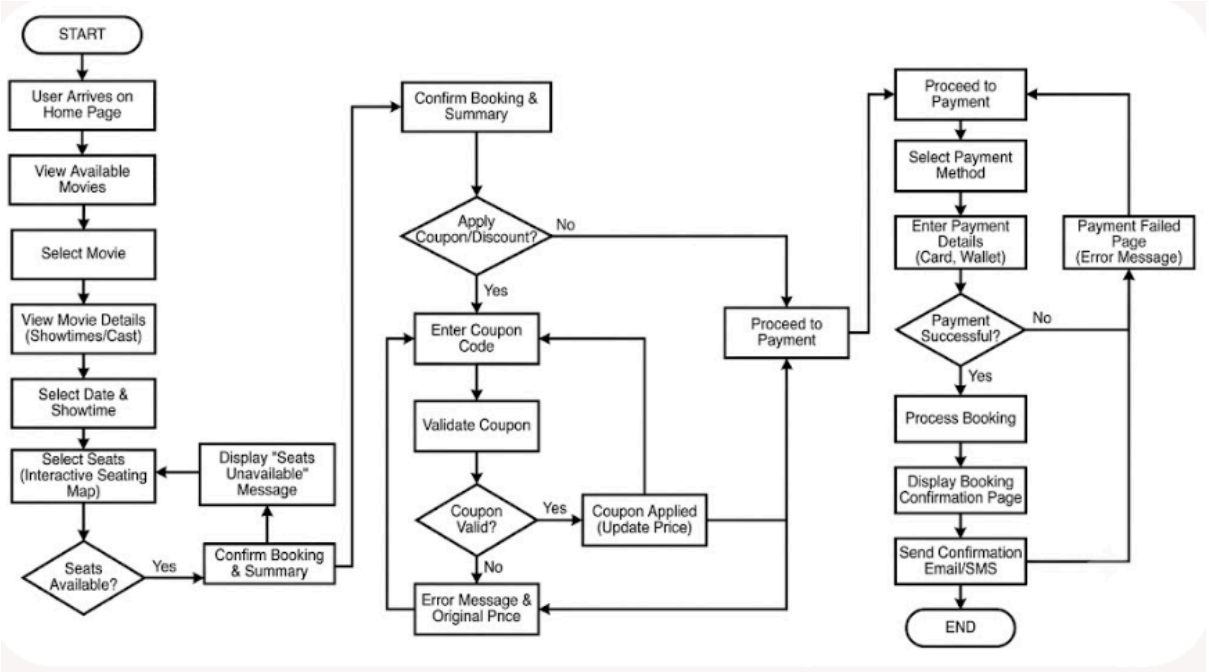
Platform Administration

- User account moderation and permission management.
- Content moderation queue for reported user reviews.
- System analytics dashboard (Active users, platform interactions, search trends).

D. Roles and Responsibilities

Role	System Responsibilities & Access Permissions
Anonymous Guest	• Browse publicly available movie listings.• Perform basic keyword searches.• View aggregate ratings and user reviews.
Registered Member	• Maintain personal user profile.• Submit, edit, or delete personal ratings and reviews.• Add titles to personal Watchlist.• Upvote or downvote other public reviews.
Movie Critic	• All Member privileges.• Verified critic badge display.• Author long-form professional reviews.• Access to advanced analytics on review engagement.
Content Manager	• Create and edit movie metadata entries.• Upload and update associated media assets.• Manage tag and genre mappings.
System Administrator	• Full operational access to platform backend.• Manage user accounts, role assignments, and bans.• Oversee audit logs, security settings, and system configurations.

E. User Flow & Explanation



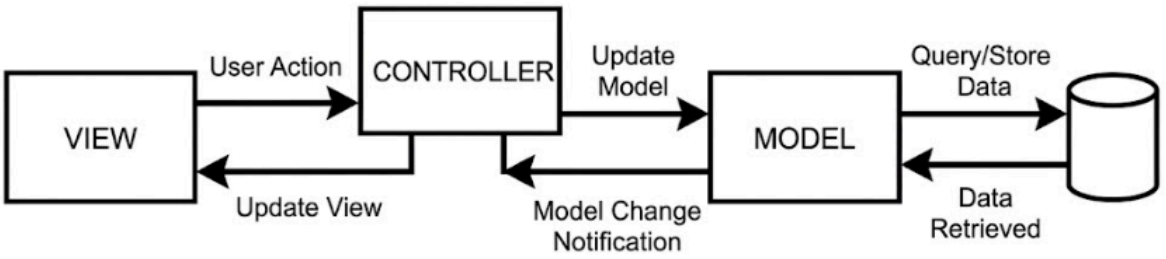
a. Authentication Flow

1. User submits login credentials via the client interface.
2. The Request Payload is encrypted and passed to the Authentication Controller.
3. Controller requests User lookup from the Service layer.
4. Password hash is validated against stored DB record.
5. Upon successful validation, a signed JWT token containing user ID and Role claims is returned to client.

b. Movie Search and Interactive Review Flow

1. **Catalog Search:**
    - User enters criteria into search bar -> Request sent to /api/v1/movies/search.
    - Controller forwards query to Search Service -> Executes indexed DB query -> Returns matched JSON records -> Client renders grid.
2. **Review Submission:**
    - User selects movie -> Detail View rendered (/movies/:id).
    - Member submits review form -> Controller validates authorization header and payload schema.
    - Review Service checks for existing entry (prevents duplicate submission).
    - Review persisted to database -> Trigger updates movies.rating_avg field.
    - Updated aggregate rating and new review entry are sent back to Client for dynamic DOM update.

F. MVC Pattern & Explanation



Components Breakdown

Model

1. Encapsulates data structures, business logic rules, and validation mechanisms.
2. Interfaces directly with the database abstraction layer (ORM/Database Client).
3. **Example:** MovieModel.js handles query execution for fetching movie metadata, updating schema validations, and calculating average ratings.

View

4. Handles client-side rendering and user user interface presentation.
5. Formats data sent by the controller into human-readable layouts (React Components, HTML/CSS templates).
6. **Example:** MovieDetailsView.jsx renders title, banner image, actor lists, and interactive review fields based on passed JSON objects.

Controller

7. Acts as an intermediary handling HTTP/HTTPS request processing.
8. Receives user input, delegates tasks to Models/Services, and determines appropriate View response or API status code.
9. **Example:** MovieController.js exposes /api/movies/:id endpoint, validates query parameters, fetches data via MovieModel, and returns 200 OK with JSON payload.